

Лекція 3.3.

Тема: Масиви у PHP

План лекції

1. Операції з масивами.
2. Сортуння масивів
3. Висновок.

Зміст лекції

Мова PHP надає безліч функцій для роботи з масивами даних. Як правило, ці функції вирішують найбільш часто зустрічаються завдання, пов'язані з обробкою масивів. У цій лекції ми розглянемо деякі з таких функцій і з їх допомогою вирішимо кілька прикладних задач. Зокрема, будуть розглянуті функції для пошуку елементів в масиві, для сортування елементів масиву, застосування створених користувачем функцій до всіх елементів масиву і розбивка масиву на подмасиви.

Масиви.

В одній з перших лекцій ми розповідали про те, як можна створити масив даних. Нагадаємо, що масив можна створити двома способами:

За допомогою конструкції `array`

```
$array_name = array("key1" => "value1", "key2" => "value2");
```

Безпосередньо задаючи значення елементів масиву

```
$array_name["key1"] = "value1";
```

Наприклад, нам потрібно зберігати список документів, які будуть видалені з бази даних. Природно зберігати його у вигляді масиву, ключем в якому буде ідентифікатор документа (його унікальний номер), а значенням - назва документа. Цей масив можна створити таким чином:

```
<?
$Del_items = array("10" => "Наука і життя", "12" => "Інформатика");
$Del_items["13"] = "Програмування на Php"; // додаємо елемент в масив
?>
```

1. Операції з масивами.

Масив - це тип даних, з даними цього типу повинні бути визначені операції. Які ж операції можна проводити з масивами? Масиви можна складати і порівнювати.

Складають масиви за допомогою стандартного оператора "+". Взагалі кажучи, цю операцію по відношенню до масивів точніше назвати об'єднанням. Якщо у нас є два масиви, *\$a* і *\$b*, то результатом їх складання (об'єднання) буде масив *\$c*, Що складається з елементів *\$a*, до яких справа дописані елементи масиву *\$b*. Причому, якщо зустрічаються однакові ключі, то в результуючий масив заноситься елемент з першого масиву, тобто з *\$a*. Таким чином, якщо складаються масиви в мові PHP, від зміни місць доданків сума змінюється.

```
<?
$a = array(
    "i" => "Інформатика",
    "m" => "Математика"
);
$b = array(
    "i" => "Історія", "m" => "Біологія",
    "ф" => "Фізика"
);
```

```

$C = $a + $b;
$D = $b + $a;
print_r($C);
/* Отримаємо: Array ([i] => Інформатика [M] => Математика [ф] => Фізика) */
print_r($D);
/* Отримаємо: Array ([i] => Історія [M] => Біологія [ф] => Фізика) */
?>

```

Приклад 1. додавання масивів

Порівнювати масиви можна, перевіряючи їх рівність чи нерівність або еквівалентність або нееквівалентність. Рівність масивів - це коли збігаються всі пари ключ/значення елементів масивів. Еквівалентність - коли крім рівності значень і ключів елементів потрібно ще, щоб елементи в обох масивах були записані в одному і тому ж порядку. Рівність значень в PHP позначається символом "=", А еквівалентність - символом "===".

```

<?
$A = array(
    "i" => "Інформатика",
    "m" => "Математика"
);
$B = array(
    "m" => "Математика",
    "I" => "Інформатика"
);
if ($A == $B) echo "Масиви рівні i";
else echo "Масиви НЕ рівні i";
if ($A === $B) echo "еквівалентні";
else echo "НЕ еквівалентні";
// отримаємо echo "Масиви рівні i НЕ еквівалентні "
?>

```

Приклад 2. порівняння масивів

Далі розглянемо ще одну важливу операцію з масивом - підрахунок кількості його елементів. Для її реалізації в PHP є спеціальна функція.

Функція count.

Ця функція обчислює число елементів в масиві. Якщо застосувати її до будь-якої іншої змінної, вона поверне 1. Виняток становить змінна типу **NULL** - *count(NULL)* є 0. Крім того, застосовуючи цю функцію до багатовимірного масиву, щоб отримати число його елементів, потрібно використовувати додатковий параметр **COUNT_RECURSIVE**.

```

<?
$del_items = array(
    "langs" => array(
        "10" => "Python", "12" => "Lisp"
    ),
    "Other" => "Інформатика"
);
echo count($del_items) . "<br>"; // виведе 2
echo count($del_items, COUNT_RECURSIVE); // виведе 4
?>

```

Приклад 3. Застосування функції count ()

Ми не будемо повторювати все, що було сказано про масивах в попередніх лекціях. У цій лекції ми розглянемо деякі вбудовані функції для роботи з масивами. І почнемо ми з функцій для пошуку значень в масиві.

Функція `in_array`

`in_array ( "шукане значення", "масив", ["Обмеження на тип"]);`

Дозволяє встановити, чи міститься в заданому масиві шукане значення. Якщо третій аргумент заданий як `true`, то в масиві потрібно знайти елемент, що співпадає з шуканим не тільки за значенням, а й за типом. Якщо шукане значення - рядок, то порівняння чутливе до регістру.

Наприклад, є масив невивчених нами мов програмування. Ми хочемо дізнатися, чи міститься в цьому масиві мову PHP. Напишемо таку програму:

```
<?php
$Langs = array(
    "Lisp", "Python", "Java",
    "PHP", "Perl"
);
if (in_array("PHP", $Langs, true))
    echo "Треба б вивчити PHP <br>";
// виведе повідомлення "Треба б вивчити PHP"
if (in_array("php", $Langs))
    echo "Треба б вивчити php <br>";
// нічого не виведе, оскільки в масиві є рядок "PHP", а не "php"
?>
```

Завданням пошуку значення цієї функції може виступати і масив.

Наприклад:

```
<?php
$Langs = array("Lisp", "Python", array("PHP", "Java"), "Perl");
if (in_array(array("PHP", "Java"), $Langs))
    echo "Треба б вивчити PHP і Java <br>";
?>
```

Функція `array_search`

Це ще одна функція для пошуку значення в масиві. На відміну від `in_array` в результаті роботи `array_search` повертає значення ключа, якщо елемент знайдений, і `brexnia` - в іншому випадку. А ось синтаксис у цих функцій однаковий:

`array_search ( "шукане значення", "масив", ["Обмеження на тип"]);`

Порівняння рядків чутливе до регістру, а якщо вказаний опціональний аргумент, то порівнюються ще і типи значень. До PHP 4.2.0, якщо шукане значення не було знайдено, ця функція повертала помилку або пусте значення `NULL`.

Приклад 4. Тепер, навпаки, нехай у нас є масив мов програмування, які ми знаємо. Причому ключем кожного елемента є номер, який вказує, яким за рахунком був вивчений цю мову.

```
<?php
$Langs = array(
    "", "Lisp", "Python", "Java",
    "PHP", "Perl"
);
if (!array_search("PHP", $Langs))
    echo "Треба б вивчити PHP <br>";
else {
    $K = array_search("PHP", $Langs);
    echo "PHP я вивчила $K-м";
}
```

```
}  
?>
```

Приклад 4. Застосування функції `array_search()`

В результаті ми отримаємо рядок:

PHP я вивчила 4-м

Очевидно, що ця функція більш функціональна, ніж `in_array()`, Оскільки ми не тільки отримуємо інформацію про те, що шуканий елемент в масиві є, але і дізнаємося, де саме в масиві він знаходиться. А що буде, якщо шуканих елементів в масиві кілька? У такому випадку функція `array_search()` поверне ключ першого зі знайдених елементів. Щоб отримати ключі всіх елементів, потрібно скористатися функцією `array_keys()`.

Функція `array_keys`

Функція `array_keys()` вибирає всі ключі масиву. Але у неї є додатковий аргумент, за допомогою якого можна отримати список ключів елементів з конкретним значенням. Синтаксис цієї функції такий:

```
array_keys ( "масив", [ "Значення для пошуку"] )
```

Функція `array_keys()` повертає як рядкові, так і числові ключі масиву, організовуючи всі значення у вигляді нового масиву з числовими індексами.

Приклад 5. Ми записали масив мов, які вивчили. Список був довгим, і деякі мови були записані кілька разів. У нас виникла підозра, що один з таких мов - Lisp. Давайте це перевіримо:

```
<?php  
$Langs =  
    array("Lisp", "Python", "Java", "PHP", "Perl", "Lisp");  
$lisp_keys = array_keys($Langs, "Lisp");  
echo "Lisp входить в масив" .count($lisp_keys) . "рази: <br>";  
foreach ($lisp_keys as $val) {  
    echo "під номером $val <br>";  
}  
?>
```

Приклад 5. Застосування функції `array_keys()`

В результаті отримаємо:

Lisp входить в масив 2 рази:

під номером 0

під номером 5

Функція `array_keys()`, Як і дві попередні, є чутливою до регістру, тобто елементів LISP в масиві вона не виявить. `array_keys()`.

Якщо є функція для отримання всіх ключів масиву, то можна припустити, що існує й третя функція для отримання всіх значень масиву. Дійсно, вона існує. це функція `array_values(Масив)`. Всі значення переданого їй масиву записуються в новий масив, проіндексований цілими числами, тобто всі ключі масиву губляться, залишаються лише значення. Але повернемося до нашого прикладу.

Отже, ми з'ясували, що мова Lisp випадково згаданий в нашому масиві двічі. Оскільки вивчити одну мову двічі не можна ("вчив, але забув" не рахується), то потрібно якось позбутися від повторюваних мов. Зробити це досить просто за допомогою функції `array_unique()`.

Функція `array_unique`

Функція `array_unique (масив)` повертає новий масив, в якому елементи, що повторюються фігурують в одному екземплярі. Таким чином, замість кількох однакових значень і їх ключів ми маємо одне значення. Який у нього буде ключ? Як з декількох ключів однакових елементів вибирається той, який буде збережений в новому масиві? Відбувається наступне. Всі елементи масива перетворюються у рядки і упорядковуються. Потім обробник запам'ятовує перший ключ для кожного значення, а інші ключі ігнорує.

Спробуємо позбутися від повторюваних мов в списку вивчених.

```
<?php
$Langs = array("Lisp", "Java", "Python", "Java", "PHP", "Perl", "Lisp");
print_r(array_unique($Langs));
?>
```

Отримаємо наступне:

```
Array ([0] => Lisp [1] => Java [2] => Python [3]
      => PHP [4] => Perl)
```

2. Сортуння масивів

Необхідність сортування даних, в тому числі і даних, що зберігаються у вигляді масивів, дуже часто виникає при вирішенні найрізноманітніших завдань. Якщо в мові Сі для того, щоб вирішити цю задачу, потрібно написати десятки рядків коду, то в PHP це робиться однією простою командою.

Функція *sort*

Функція *sort* має наступний синтаксис

sort (масив [, прапори])

і сортує масив, тобто впорядковує його значення по зростанню. Ця функція видаляє всі існуючі в масиві ключі, замінюючи їх числовими індексами, відповідними новим порядком елементів. У разі успішного завершення роботи вона повертає *true*, Інакше - *false*.

Приклад 6. Нехай у нас є два масиви: ціни товарів - їх назви і, навпаки, назви товарів - їх ціни. Впорядкуємо ці масиви по зростанню:

```
<?
$items = array(10 => "хліб", 20 => "молоко", 30 => "бутерброд");
sort($items);
// рядки сортуються в алфавітному
// порядку, ключі губляться
print_r($items);

$rev_items = array("хліб" => 10, "Бутерброд" => 30, "молоко" => 20);
sort($rev_items);
// числа сортуються за зростанням,
// ключі губляться
print_r($rev_items);
?>
```

Приклад 6. Застосування функції *sort* ()

отримаємо:

```
Array ([0] => бутерброд [1] =>
      молоко [2] => хліб)
Array ([0] => 10 [1] => 20 [2] => 30)
```

Як додатковий аргумент може використовуватися одна з наступних констант:

SORT_REGULAR- порівнювати елементи масиву звичайним чином;

SORT_NUMERIC- порівнювати елементи масиву як числа;

SORT_STRING- порівнювати елементи масиву як рядки.

Функції *asort*, *rsort*, *arsort*

Якщо потрібно зберігати індекси елементів масиву після сортування, то потрібно використовувати функцію *asort* (масив [, прапори]). Якщо необхідно впорядкувати масив в зворотному порядку, тобто від найбільшого значення до найменшого, то можна задіяти функцію *rsort* (масив [, прапори]). А якщо при цьому потрібно ще й зберегти значення ключів, то слід використовувати функцію *arsort* (масив [, прапори]). Як ви, напевно, помітили синтаксис у цих функцій абсолютно такий же, як у функції *sort*. Відповідно і

значення прапорів можуть бути такими ж, як у *sort*: *SORT_REGULAR*, *SORT_NUMERIC*, *SORT_STRING*.

```
<?php
$books = array("Пушкін" => "Руслан і Людмила", "Толстой" => "Війна і мир", "Лермонтов" => "Герой нашого часу");
asort($books);
// сортуємо масив,
// зберігаючи значення ключів
print_r($books);
echo "<br>";
rsort($books);
// сортуємо масив в зворотному порядку,
// ключі будуть замінені
print_r($books);
?>
```

Приклад 7. Застосування функцій *asort*, *rsort*, *arsort*

В результаті роботи цього скрипта одержимо:

```
Array ([Толстой] => Війна і мир
      [Лермонтов] => Герой нашого часу
      [Пушкін] => Руслан і Людмила)
Array ([0] => Руслан і Людмила
      [1] => Герой нашого часу
      [2] => Війна і мир)
```

Приклад 8. Припустимо, ми створюємо каталог описів документів. У кожного документа є автор, назва, дата публікації та короткий зміст. Ми вже не раз відображали опису, складені з цих характеристик. Щоразу порядок відображення цих елементів залежав від створеної нами програми. Тепер же ми хочемо мати можливість змінювати порядок відображення елементів за бажанням користувача. Складемо для цього наступну форму:

```
<Form action=task.php>
  <Table border=1>
    <Tr>
      <td> Назва </td>
      <td> <input type=text name=title size=5> </td>
    </tr>
    <Tr>
      <td> Короткий зміст </td>
      <td> <input type=text name=description size=5> </td>
    </tr>
    <Tr>
      <td> Автор </td>
      <td> <input type=text name=author size=5> </td>
    </tr>
    <Tr>
      <td> Дата публікації </td>
      <td> <input type=text name=published size=5> </td>
    </tr>
  </ Table>
  <input type=submit value="Відправити">
</Form>
```

Приклад 8a. Форма для прикладу 8

Будемо впорядковувати дані, передані цією формою, по спадаючій їх значень, зберігаючи при цьому значення ключів. Для цього зручно скористатися функцією *arsort()*. Оскільки нам важливий тільки новий порядок елементів, збережемо в новому масиві ключі вихідного масиву в потрібному порядку. Ми зберігаємо ключі вихідного масиву, оскільки вони є іменами елементів, з яких конструюється опис документа, а пам'ятати їх важливо. Отже, отримуємо такий скрипт:

```
<?php
print_r($_GET);
echo "<br>";
arsort($_GET);
// сортуємо масив в зворотному порядку,
// зберігаючи ключі
print_r($_GET);
echo "<br>";
$ordered_names = array_keys($_GET);
// складаємо новий масив
foreach ($ordered_names as $key => $val)
    echo "$key: $val <br>";
// виводимо елементи нового масиву
?>
```

Приклад 8b. Програма обробки форми з прикладу 8 Сортування масиву по ключам

Очевидно, що може виникнути необхідність у сортуванні масиву за значеннями ключів. Наприклад, якщо у нас є масив даних по книгах, як у наведеному вище прикладі, то цілком ймовірно, що ми захочемо впорядкувати книги по іменах авторів. Для цього в PHP також не потрібно писати багато рядків коду - можна просто скористатися функцією *ksort()* для сортування по зростанню (прямий порядок сортування) або *krsort()* - для сортування по спадаючій (зворотний порядок сортування). Синтаксис цих функцій знову ж аналогічний синтаксису функції *sort()*.

```
<?php
$books = array("Пушкін" => "Руслан і Людмила", "Толстой" => "Війна і мир", "Лермонтов" => "Герой нашого часу");
ksort($books);
// сортуємо масив,
// зберігаючи значення ключів
print_r($books);
?>
```

Приклад 9. Сортування масиву по ключам отримаємо:

```
Array ([Лермонтов] => Герой нашого часу
      [Пушкін] => Руслан і Людмила
      [Толстой] => Війна і мир)
```

Виділення під масиву Функція *array_slice*

Оскільки масив - це набір елементів, цілком ймовірно, буде потрібно виділити з нього який-небудь піднабір. У PHP для цих цілей є функція *array_slice*. Її синтаксис такий:
array_slice (масив, номер_елемента [, довжина])

Ця функція виділяє підмасив довжини *довжина* в масиві *масив*, Починаючи з елемента, номер якого заданий параметром *номер_елемента*. Позитивний *номер_елемента* вказує на порядковий номер елемента щодо початку масиву, негативний - на номер елемента з кінця масиву.


```

<?php
$arr = array (1,2,3,4,5);
$sub_arr = array_slice ($arr, 2);
print_r ($sub_arr);
/*
виведе Array ([0] => 3 [1] => 4 [2] => 5),
тобто підмасив, що складається з елементів
3, 4, 5 */
$sub_arr = array_slice ($arr, -2);
print_r ($sub_arr);
    // виведе Array ([0] => 4 [1] => 5),
    // тобто підмасив, з елементів 4, 5
?>

```

Приклад 13. Використання функції array_slice ()

Якщо задати параметр **довжина** при використанні *array_slice*, То буде виділено підмасивів, що має рівно стільки елементів, скільки задано цим параметром. Довжину можна вказувати і негативну. В цьому випадку інтерпретатор видалить з кінця масиву число елементів, рівне модулю параметра **довжина**.

```

<?php
$arr = array(1, 2, 3, 4, 5);
$Sub_arr = array_slice($arr, 2, 2);
// містить масив з елементів 3, 4
$Sub = array_slice($arr, -3, 2);
// теж містить масив з елементів 3, 4
$Sub1 = array_slice($arr, 0, -1);
// містить масив з
// елементів 1, 2, 3, 4
$Sub2 = array_slice($arr, -4, -2);
// містить масив з елементів 2, 3
?>

```

Приклад 14. Використання функції array_slice (). Варіант 2

Функція array_chunk

Є ще одна функція, схожа на *array_slice()* - це *array_chunk()*. Вона розбиває масив на кілька підмасивів заданої довжини. Синтаксис її такий:

array_chunk (масив, розмір [, зберігати_ключі])

В результаті роботи *array_chunk ()* повертає багатовимірний масив, елементи якого являють собою отримані підмасиви. Якщо задати параметр зберігати ключі як **true**, То при розбитті будуть збережені ключі вихідного масиву. В іншому випадку ключі елементів замінюються числовими індексами, які починаються з нуля.

Приклад 15. У нас є список запрошених, оформлений у вигляді масиву їх прізвищ. У нас є столики на три персони. Тому потрібно розподілити всіх запрошених по трое.

```

<?php
$Persons = array ("Іванов", "Петров", "Сидорова", "Зайцева", "Волкова");
$Triples = array_chunk ($persons, 3);
    // ділимо масив на підмасиви
    // по три елементи
foreach ($triples as $k => $table) {
    // виводимо отримані трійки
    echo "За столиком номер $ k сидять: <ul>";
    foreach ($table as $pers)
        echo "<li> $pers";
}

```



```

    echo "</ ul>";
}
?>

```

Приклад 15. Використання функції `array_chunk ()`

В результаті отримаємо:

за столиком номер 0 сидять:

- Іванов
- Петров
- Сидорова

за столиком номер 1 сидять:

- Зайцева
- Волкова

Сума елементів масиву

У цьому розділі ми познайомимося з функцією, що обчислює суму всіх елементів масиву. Сама задача обчислення суми значень масиву гранично проста. Але навіть писати зайвий раз один і той же код, якщо можна скористатися спеціально створеною і завжди доступною функцією. Функція ця називається, як можна здогадатися, *`array_sum ()`*. І як параметр їй передається тільки ім'я масиву, суму значень елементів якого потрібно обчислити.

Як приклад використання цієї функції наведемо рішення більш складного завдання, ніж просто обчислення суми елементів. Цей приклад також ілюструє застосування функції *`array_slice ()`*, яку ми обговорювали трохи раніше.

Приклад 16. Нехай дано масив натуральних чисел. Потрібно знайти в ньому таке число, що сума елементів праворуч від нього дорівнює сумі елементів зліва від нього.

```

<?php
// масив задається функцією array
$arr = array(2, 1, 3, 4, 5, 6, 4);
// перебираємо кожен елемент масиву $ arr. Всередині циклу поточний ключ масиву
// міститься в змінній $ k, поточне значення - в змінній $ val
foreach ($arr as $k => $val) {
    $p = $k + 1;
    // синтаксис array array_slice(array array, int offset [, int length])
    // array_slice виділяє підмасив довжини length в масиві array, починаючи з е
    // лементу offset.
    $out_next = array_slice($arr, $p);
    // отримуємо масив елементів, йдуть після поточного
    $out_prev = array_slice($arr, 0, $k);
    // отримуємо масив елементів, йдуть перед поточним функція mixed array_sum (a
    rray array)
    // підраховує суму елементів масиву array
    $next_sum = array_sum($out_next);
    $prev_sum = array_sum($out_prev);
    // якщо сума елементів до поточного дорівнює сумі елементів після, то виводим
    o
    // значення поточного елемента
    if ($next_sum == $prev_sum)
        echo "value: $val";
    // можна подивитися, що представляють собою розглянуті масиви на кожному кроц
    i
    // print_r($out_next); echo "<br>";
    // print_r($out_prev);

```

```

    // echo "$next_sum, $prev_sum <br>";
    echo "<hr>";
}
?>

```

Приклад 16. Програма пошуку числа, такого що сума елементів праворуч від нього дорівнює сумі елементів зліва від нього

Контрольні питання.

1. **Створення масивів:** Як створити індексований та асоціативний масив? Опишіть синтаксис `array()` та короткий синтаксис `[]` (якщо згадується).
2. **Додавання та видалення елементів:** Як додати новий елемент в кінець масиву (`$arr[] = value`)? Як видалити елемент або весь масив за допомогою `unset()`?
3. **Перебір масиву:** Як працює цикл `foreach`? Опишіть два варіанти синтаксису: `foreach($arr as $value)` та `foreach($arr as $key => $value)`.
4. **Підрахунок елементів:** Яка функція використовується для визначення кількості елементів у масиві (`count()`)?
5. **Пошук у масиві:** Як перевірити наявність значення в масиві (`in_array()`) або наявність ключа (`array_key_exists()`)? Як знайти ключ за значенням (`array_search()`)?
6. **Сортування масивів:** Опишіть функції сортування: `sort()` (за зростанням), `rsort()` (за спаданням), `asort()` (асоціативне за значенням), `ksort()` (за ключем).
7. **Злиття масивів:** Як об'єднати два або більше масивів в один за допомогою функції `array_merge()` або оператора `+`? У чому різниця?
8. **Зріз масиву:** Як отримати частину масиву за допомогою `array_slice()`? Які параметри приймає ця функція (`offset`, `length`)?
9. **Отримання ключів та значень:** Як отримати всі ключі масиву (`array_keys()`) або всі значення (`array_values()`) окремим списком?
10. **Унікальні значення:** Як видалити дубльовані значення з масиву за допомогою функції `array_unique()`?
11. **Розбиття масиву:** Як розділити масив на частини (чанки) за допомогою `array_chunk()`? Для чого це може знадобитися при виводі даних?
12. **Математичні операції:** Як знайти суму всіх числових елементів масиву (`array_sum()`)?